

Proyecto 2. EcoCheems

Equipo: SyntaxError

Integrantes:

Centeno Lopez Shani Samantha	319032680
Hernández Barrios Carlos	321297374
Mendoza Romero Fabián	321269568

Proyecto: videojuego educativo para la enseñanza de separación de residuos sólidos en niños entre 5 y 10 años.

Nombre: EcoCheems

Problemática: El manejo incorrecto de los residuos sólidos es un problema muy marcado en la actualidad. Según datos de SEMARNAT, una persona en promedio genera 1.4 kg de residuos al día y gran parte de estos residuos terminan en rellenos sanitarios o ecosistemas naturales debido a la falta de cultura de reciclaje.

Uno de los puntos más débiles en el reciclaje es la separación desde el hogar y ante esta situación, es que los niños, a quienes se les puede inculcar el hábito de separar residuos, representan una gran oportunidad para generar un cambio en la sociedad. Sin embargo, los métodos de enseñanza tradicionales como folletos, carteles o pláticas suelen ser poco atractivos y los niños no logran retener esta información para ponerla en práctica después, así que aquí es donde se encuentra una oportunidad para satisfacer esta necesidad de adquirir el hábito de separar nuestros residuos sólidos desde el hogar.

Nuestra propuesta de solución para esta necesidad es: desarrollar un videojuego interactivo donde los niños entre 5 y 10 años clasifiquen correctamente diferentes residuos sólidos: orgánicos, papel, cartón, plástico, vidrio, metal y no reciclables. El residuo aparece en pantalla sobre diferentes tipos de contenedores y el usuario selecciona el contenedor que considere adecuado para clasificar dicho residuo. El juego incluye: puntos que se van acumulando conforme se vaya clasificando correctamente el residuo sólido, vidas que se van perdiendo conforme el usuario se equivoque al clasificar, niveles de dificultad progresiva (fácil, medio, difícil) en donde se modifica el número de vidas que tiene el usuario y desbloqueo de logros.

El alcance del proyecto es una simulación de residuos sólidos a clasificar mediante imágenes, un sistema de puntuación y vidas, además de que es funcional para sólo un jugador.

Patrones utilizados:

- **MVC.** Nos ayuda a separar la aplicación en tres componentes: modelo que tendrá la lógica del juego (`EcoCheemsModelo.java`), vista que contendrá la interfaz gráfica (`EcoCheemsVista.java`), y el controlador que manejará los eventos del usuario (`EcoCheemsControlador.java`).

En el videojuego, la vista cambia constantemente, al separarla del modelo, podremos modificar la apariencia sin modificar la lógica del juego, mientras que el controlador funciona como puente entre ambos, recibe el clic del usuario, válida si es correcta la clasificación y actualiza tanto el modelo como la vista.

MVC nos permite que el mismo modelo pueda funcionar con una interfaz de consola o con una interfaz gráfica con JavaFX sin modificaciones; también nos permite darle mantenimiento fácilmente sin afectar la lógica del juego y se puede probar el modelo sin la necesidad de la interfaz gráfica.

- **Strategy.** Definir una familia de algoritmos encapsulados e intercambiables permite que el juego varíe independientemente de quién lo usa, lo que nos ayuda a determinar la dificultad del juego de diferentes maneras. Como el juego debe de ser adaptado para niños de distintas edades y niveles de conocimiento, Strategy nos permite cambiar la cantidad de vidas iniciales, el tiempo límite por ronda, la cantidad de residuos por nivel y los puntos por aciertos o errores.

El patrón strategy encapsula cada dificultad en una clase separada: `DificultadFacil`, `DificultadMedia`, `DificultadDifícil`, que implementan a la interfaz `EstrategiaDificultad`, lo que permite cambiar la dificultad en tiempo de ejecución sin modificar el modelo y agregar nuevas dificultades sin afectar el código existente. De esta manera agregar un nuevo nivel implica únicamente crear una nueva clase que implemente la interfaz de dificultad del juego

- **Factory Method.** Define una interfaz para crear objetos, pero permite que las subclases decidan qué clase instanciar. El juego necesita generar residuos de diferentes tipos (botella de plástico, cáscara de plátano, etc.) de manera aleatoria durante las rondas.

El patrón Factory Method centraliza la creación de objetos `ObjetoBasura` en `BasuraFactory`, permite agregar nuevos tipos de residuos sólo creando nuevas subclases de `ObjetoBasura` y actualizando la fábrica; también aísla al modelo y al controlador de los detalles concretos de

creación de residuos y al incluir un método `generarBasuraAleatoria()` que retorna un objeto `ObjetoBasura` de tipo aleatorio, nos ayuda a mantener la variedad en cada partida sin complicar el resto del código.

- **Observer.** El patrón observer define una dependencia uno a muchos, así que cuando un objeto cambia de estado todos sus suscriptores son notificados automáticamente. Durante una partida ocurren varias acciones: actualice el contador de logros, se verifique el récord, se actualice el puntaje y se reproduzca un sonido especial, así que en lugar de que el modelo mande a llamar a cada uno, podemos utilizar observer para que estas acciones se “suscriban” al modelo y reaccionen de forma automática; esto nos puede ayudar a la extensión en el futuro sin modificar lo existente.

El sujeto es `EcoCheemsModelo`, el observador es una clase abstracta `EventoJuegoObserver`, los observadores concretos son `LogroObserver` que detecta rachas de 5,10,15 aciertos y desbloquea logros, `EstadísticasObserver` registra los aciertos, errores, la racha máxima y los puntos totales; `SonidoObserver` reproduce efectos de sonido (en consola de forma simulada); `VistaObserver` actualiza la interfaz gráfica en tiempo real y `RecordObserver` gestiona y reacciona a los nuevos récords.

La ventaja de usar una clase abstracta es que cada observador sólo necesita implementar los métodos que le interesan. Funciona como un complemento a MVC, ya que mientras MVC sincroniza el modelo con la vista de manera general, el patrón observer permite reaccionar a eventos específicos del juego sin tener que adaptar el modelo a los observadores concretos.

- **State.** El juego maneja las vidas del jugador como un estado que cambia con cada error, lo que nos ayuda a evitar grandes bloques if-else o switch en el Controlador. Cada estado representa un número de vidas distinto, donde cada uno es una clase independiente que encapsula su propia lógica y de esta manera, agregar un nuevo estado no causaría problema, simplemente se crea su clase sin modificar lo ya existente. Las clases son: `EstadoVidas` (interfaz), `Estado3Vidas`, `Estado2Vidas`, `Estado1Vida` (estados concretos). Cada estado sabe cuál es el siguiente al perder una vida y el modelo solo delega en el estado actual. Es importante mencionar que state no se utiliza para cambiar la pantalla, ya que de eso se encarga MVC, solo cambia el comportamiento dentro de la partida relacionado con las vidas.

Al momento de realizar este proyecto, y como se trata de un programa pensado para niños, pensamos que era una buena idea aplicarle una interfaz gráfica con el uso de herramientas como `javafx` y `Scene Builder`.

Nuestra ingenuidad nos hizo creer que esto no sería tan complicado, no obstante, en el camino hubo bastante obstáculos que nos hicieron perder algo de tiempo en el proceso.

También queríamos lograr una interfaz algo más estética que fuera con la temática del reciclaje, pero de igual manera, el tiempo fue nuestro peor enemigo y no lo hicimos ver como teníamos planeado en un inicio.

A pesar de todo esto, creemos que nuestro proyecto es funcional y cumple con su propósito y todos los pequeños cambios que teníamos en mente implementar desde un inicio, se pueden hacer en un trabajo a futuro, ya que tenemos algo sólido y funcional. También como trabajo a futuro se puede implementar que sea un juego tipo “kahoot” para que más niños puedan jugar y competir con sus amigos, para que, en el salón de clases, los profesores puedan implementar en sus clases el juego entre todos sus alumnos y que todos aprendan de manera divertida.

Para el diagrama de estados, por temas de tiempo, no logramos implementar más funcionalidades dentro de la partida, ya que el diagrama nos muestra el estado de las vidas, motivo por el cual, no hay botón de reiniciar, pausar, salir durante la partida hasta que se termina el juego. Por eso el diagrama se puede ver “simple”, ya que para cambiar de estado es necesario no acertar en la clasificación, mientras haya un acierto, se mantendrá en su estado actual.

Los sonidos implementados en el juego fueron sacados de la página freesound.org y las imágenes fueron sacadas de fotos de Google. Muy probablemente estos elementos tengan copyright (a excepción de los elementos de freesound) pero este proyecto no tiene fines de lucro, así que no vemos el problema en utilizarlos.

Para ejecutar el usuario se tendrá que posicionar en la carpeta titulada “Proyecto02_SyntaxError”, una vez dentro debe ejecutar el comando: `mvn javafx:run`

Las versiones de java usadas para la elaboración de la práctica fueron: 21, 24, 25.